Light

Programming — Data Structures

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Arrays (1D)
2. Two-dimensional arrays
3. Records
Worked examples
Quick reference
Typical pitfalls

AQA GCSE Computer Science — **Programming: Data Structures (8525 Paper 1)**. This topic covers how to organise and store collections of related data using arrays (one-dimensional and two-dimensional) and records. You need to understand how to create, access, and manipulate these structures in your programs.

Exam questions typically ask you to **define** what a data structure is, **complete** code that uses arrays or records, **trace** through algorithms that process data structures, and **write** programs that store and retrieve data from arrays and records.

Why data structures matter

A **data structure** is an organised collection of related data values. Instead of creating many separate variables, data structures let you group related data together, making your programs more organised and efficient.

Consider storing the names of 100 students. Without data structures, you would need 100 separate variables (`student1`, `student2`, ... `student100`). With an array, you can store all 100 names in a single structure and access any name using its position number (index).

At GCSE, you need to understand three types of data structure: **one-dimensional arrays** (lists), **two-dimensional arrays** (grids/tables), and **records** (grouping

different data types together).

What you must know

This checklist covers the key exam-visible concepts for data structures. You should be able to explain these concepts and write code that uses them.

- **Define** what is meant by a data structure.
- **Use arrays** (one-dimensional) in the design of solutions to simple problems.
- **Use two-dimensional arrays** in the design of solutions to simple problems.
- **Use records** in the design of solutions to simple problems.
- **Access** individual elements in arrays using index values.
- **Create** record definitions with appropriate field names and data types.

> **Specification note.** AQA Paper 1 assesses data structures through code completion, trace tables, and programming tasks. Exam questions use AQA pseudo-code, but you can write answers in Python. Only one-dimensional and two-dimensional arrays are required — no other complex data structures.

Toolkit

Key terminology

Term	Definition	Example
Data structure	An organised collection of related data values	An array of student names
Array	A data structure that stores multiple values of the same data type, accessed by index	`scores = [85, 92, 78, 95]`
Index	The position of an element in an array (starting from 0)	`scores[0]` gives 85
Element	A single item stored in an array	92 is an element in the scores array
Two-dimensional array	An array of arrays, organised in rows and columns like a grid	A seating plan or game board
Record	A data structure that groups related data of different types under named fields	A student record with name, age, and grade
Field	A named item of data within a record	`student.name`

Data structures comparison

Structure	Best used for	Access method	Example use
1D Array	A list of similar items	Single index: `array[i]`	List of test scores
2D Array	Grid or table of data	Row and column: `array[row][col]`	Game board, seating plan
Record	Grouping different data about one entity	Field name: `record.field`	Student details, product info

1. Arrays (one-dimensional)

1.1 What is an array?

An **array** is a data structure that stores a collection of values of the same data type. Each value is stored at a specific position called an **index**. In most programming languages (including Python), array indices start at **0**, not 1.

```
ARRAY STRUCTURE — scores = [85, 92, 78, 95, 88]

Index:     0     1     2     3     4
         +-----+-----+-----+-----+-----+
Value:   |  85 |  92 |  78 |  95 |  88 |
         +-----+-----+-----+-----+-----+

scores[0] = 85    (first element)
scores[2] = 78    (third element)
scores[4] = 88    (last element, index = length - 1)

Length of array: 5 elements
```

1.2 Creating arrays in Python

In Python, arrays are implemented using **lists**. You create a list by placing values inside square brackets, separated by commas.

```python
# Creating arrays (lists) in Python

# An array of integers
scores = [85, 92, 78, 95, 88]

# An array of strings
```

```
names = ["Alice", "Bob", "Charlie", "Diana"]

# An empty array (to add items later)
empty_list = []

# An array with a set size, filled with zeros
grid = [0] * 10  # Creates [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

1.3 Accessing and modifying elements

You access individual elements using their index in square brackets. Remember that the first element is at index 0.

```
# Accessing elements
scores = [85, 92, 78, 95, 88]

first_score = scores[0]   # 85
third_score = scores[2]   # 78
last_score = scores[4]    # 88

# Modifying elements
scores[2] = 82  # Change 78 to 82
# scores is now [85, 92, 82, 95, 88]

# Getting the length of an array
size = len(scores)  # 5

# Accessing the last element using length
last = scores[len(scores) - 1]  # 88
```

1.4 Iterating through arrays

To process every element in an array, you use a loop. There are two common approaches: using the element directly, or using an index.

```
# Method 1: Loop through elements directly
names = ["Alice", "Bob", "Charlie"]
for name in names:
    print(name)
# Output: Alice, Bob, Charlie (on separate lines)
```

```python
# Method 2: Loop using index (when you need the position)
scores = [85, 92, 78]
for i in range(len(scores)):
    print("Score", i, "is", scores[i])
# Output: Score 0 is 85, Score 1 is 92, Score 2 is 78
```

1.5 Common array operations

Operation	Python code	Result
Get length	`len(scores)`	Number of elements
Access element	`scores[2]`	Element at index 2
Change element	`scores[2] = 80`	Updates index 2 to 80
Add to end	`scores.append(90)`	Adds 90 to end of array
Remove last	`scores.pop()`	Removes and returns last element
Find maximum	`max(scores)`	Largest value in array
Find minimum	`min(scores)`	Smallest value in array
Calculate sum	`sum(scores)`	Total of all values

1.6 Common array algorithms

Many programming tasks involve processing arrays. Here are some common algorithms you should be able to write.

Counting items that meet a condition:

```python
# Count how many scores are above 70
scores = [85, 62, 78, 95, 55, 88]
count = 0
for i in range(len(scores)):
    if scores[i] > 70:
        count = count + 1
print("Scores above 70:", count)  # Output: 4
```

Calculating the average:

```python
# Calculate average of all scores
scores = [85, 62, 78, 95, 55, 88]
total = 0
for i in range(len(scores)):
    total = total + scores[i]
average = total / len(scores)
print("Average:", average)  # Output: 77.17
```

Searching for a value:

```python
# Search for a specific name
names = ["Alice", "Bob", "Charlie", "Diana"]
search = "Charlie"
found = False
position = -1

for i in range(len(names)):
    if names[i] == search:
        found = True
        position = i

if found:
    print(search, "found at position", position)
else:
    print(search, "not found")
# Output: Charlie found at position 2
```

Exam technique: Array algorithms. Many exam questions ask you to write algorithms that process arrays. Common tasks include: finding the total or average, finding the largest or smallest value, counting items that meet a condition, or searching for a specific value. Always think about what happens at each index.

2. Two-dimensional arrays

2.1 What is a 2D array?

A **two-dimensional array** (2D array) is an array of arrays, organised in **rows** and **columns** like a grid or table. You access elements using two indices: the row number and the column number.

```
2D ARRAY STRUCTURE — A 3×4 grid (3 rows, 4 columns)

              Column 0  Column 1  Column 2  Column 3
            +---------+---------+---------+---------+
Row 0       |    5    |    8    |    3    |    1    |
            +---------+---------+---------+---------+
Row 1       |    2    |    9    |    4    |    7    |
            +---------+---------+---------+---------+
Row 2       |    6    |    0    |    1    |    8    |
            +---------+---------+---------+---------+

grid[0][0] = 5     (top-left corner)
grid[1][2] = 4     (row 1, column 2)
grid[2][3] = 8     (bottom-right corner)

Access pattern: grid[row][column]
```

2.2 Creating 2D arrays in Python

In Python, a 2D array is a list of lists. Each inner list represents a row.

```python
# Creating a 2D array (list of lists)
grid = [
    [5, 8, 3, 1],   # Row 0
    [2, 9, 4, 7],   # Row 1
    [6, 0, 1, 8]    # Row 2
]

# A 3x3 game board (noughts and crosses)
board = [
    [" ", " ", " "],
    [" ", " ", " "],
    [" ", " ", " "]
]

# Creating a 2D array filled with zeros
rows = 3
```

```
cols = 4
empty_grid = [[0 for j in range(cols)] for i in range(rows)]
```

2.3 Accessing and modifying 2D array elements

Use two indices in square brackets: the first for the row, the second for the column.

```
grid = [
    [5, 8, 3, 1],
    [2, 9, 4, 7],
    [6, 0, 1, 8]
]

# Accessing elements
top_left = grid[0][0]      # 5
middle = grid[1][2]        # 4
bottom_right = grid[2][3]  # 8

# Modifying elements
grid[1][2] = 99  # Change row 1, column 2 to 99

# Getting dimensions
num_rows = len(grid)        # 3
num_cols = len(grid[0])     # 4
```

2.4 Iterating through 2D arrays

To process every element in a 2D array, you use **nested loops** — an outer loop for rows and an inner loop for columns.

```
grid = [
    [5, 8, 3],
    [2, 9, 4],
    [6, 0, 1]
]

# Print all elements row by row
for row in range(len(grid)):
    for col in range(len(grid[0])):
        print(grid[row][col], end=" ")
```

```python
    print()  # New line after each row

# Output:
# 5 8 3
# 2 9 4
# 6 0 1

# Calculate sum of all elements
total = 0
for row in range(len(grid)):
    for col in range(len(grid[0])):
        total = total + grid[row][col]
print(total)  # 38
```

```
NESTED LOOP EXECUTION — Processing a 2D array

grid = [[A, B, C],
        [D, E, F]]

Loop execution order:
+-----+-----+-----+
| 1st | 2nd | 3rd |   row = 0
+-----+-----+-----+
| 4th | 5th | 6th |   row = 1
+-----+-----+-----+
  col=0 col=1 col=2

Step 1: row=0, col=0 → grid[0][0] = A
Step 2: row=0, col=1 → grid[0][1] = B
Step 3: row=0, col=2 → grid[0][2] = C
Step 4: row=1, col=0 → grid[1][0] = D
Step 5: row=1, col=1 → grid[1][1] = E
Step 6: row=1, col=2 → grid[1][2] = F
```

Exam technique: 2D array questions. In exam questions, 2D arrays often represent game boards, seating plans, or grids of data. Pay careful attention to the order of indices — always `array[row][column]`. Draw the grid on paper if it helps you visualise which element is being accessed.

2.5 Practical 2D array examples

Here are some common tasks you might need to perform with 2D arrays.

Finding a value in a grid:

```
# Search for value 0 (blank space) in a game board
board = [
    [1, 2, 3],
    [4, 0, 5],
    [6, 7, 8]
]

for row in range(len(board)):
    for col in range(len(board[0])):
        if board[row][col] == 0:
            print("Blank at row", row, "column", col)
# Output: Blank at row 1 column 1
```

Counting specific values:

```
# Count how many X marks on a noughts and crosses board
board = [
    ["X", " ", "O"],
    [" ", "X", " "],
    ["O", " ", "X"]
]

x_count = 0
for row in range(3):
    for col in range(3):
        if board[row][col] == "X":
            x_count = x_count + 1
print("X appears", x_count, "times")  # Output: 3
```

Displaying a grid:

```python
# Display a seating plan
seats = [
    ["A1", "A2", "A3"],
    ["B1", "B2", "B3"],
    ["C1", "C2", "C3"]
]

for row in range(len(seats)):
    for col in range(len(seats[0])):
        print(seats[row][col], end=" ")
    print()  # New line after each row

# Output:
# A1 A2 A3
# B1 B2 B3
# C1 C2 C3
```

3. Records

3.1 What is a record?

A **record** is a data structure that groups together related data of **different types** under named **fields**. Unlike arrays where all elements are the same type, a record can contain a mix of strings, integers, real numbers, and booleans.

Records are ideal for representing real-world entities like a person, a product, or a book — where you need to store several different pieces of information together.

```
RECORD STRUCTURE — A Car record

+------------------+------------------+
|     Field        |      Value       |
+------------------+------------------+
| make    (String) |     "Ford"       |
| model   (String) |     "Focus"      |
| reg     (String) |     "AB12 CDE"   |
| price   (Real)   |     15999.99     |
| noOfDoors (Int)  |     5            |
+------------------+------------------+

Each field has:
  • A name (like a variable name)
```

```
    • A data type
    • A value
```

3.2 Defining records in AQA pseudo-code

In AQA pseudo-code, you define a record structure using `RECORD` and `ENDRECORD`. This is the format you will see in exam questions.

```
AQA PSEUDO-CODE — Record definition

RECORD Car
    make : String
    model : String
    reg : String
    price : Real
    noOfDoors : Integer
ENDRECORD

Creating a record variable:
myCar ← Car("Ford", "Focus", "AB12 CDE", 15999.99, 5)

Accessing fields:
OUTPUT myCar.make        → "Ford"
OUTPUT myCar.price       → 15999.99
myCar.price ← 14500.00   → Updates the price
```

3.3 Implementing records in Python

Python doesn't have a built-in record type, but you can use a **dictionary** to achieve the same result. Each key in the dictionary acts as a field name.

```python
# Creating a record using a dictionary
car = {
    "make": "Ford",
    "model": "Focus",
    "reg": "AB12 CDE",
    "price": 15999.99,
    "noOfDoors": 5
```

```
}

# Accessing fields
print(car["make"])      # Ford
print(car["price"])     # 15999.99

# Modifying fields
car["price"] = 14500.00

# Creating multiple records
car1 = {"make": "Ford", "model": "Focus", "price": 15999.99}
car2 = {"make": "Toyota", "model": "Corolla", "price": 18500.00}
```

3.4 Working with records — practical examples

```
# Book record example
book1 = {
    "bookName": "The Book Thief",
    "author": "M Zusak",
    "price": 9.99
}

book2 = {
    "bookName": "Divergent",
    "author": "V Roth",
    "price": 6.55
}

# Comparing prices to find the most expensive
if book1["price"] > book2["price"]:
    print(book1["bookName"])
elif book2["price"] > book1["price"]:
    print(book2["bookName"])
else:
    print("Neither")

# Output: The Book Thief
```

3.5 Arrays of records

You can combine arrays and records to store a collection of records — for example, a list of all books in a library or all students in a class.

```
# An array of student records
students = [
    {"name": "Alice", "age": 16, "grade": "A"},
    {"name": "Bob", "age": 15, "grade": "B"},
    {"name": "Charlie", "age": 16, "grade": "A"}
]

# Accessing a specific student
print(students[0]["name"])  # Alice

# Looping through all students
for student in students:
    print(student["name"], "got grade", student["grade"])

# Counting students with grade A
count = 0
for student in students:
    if student["grade"] == "A":
        count = count + 1
print("Number with grade A:", count)  # 2
```

Exam technique: Record questions. AQA exam questions often provide a partially completed record definition and ask you to fill in the blanks. Look carefully at the field names and data types — make sure strings are marked as String, whole numbers as Integer, and numbers with decimals as Real. Watch for the correct syntax: `fieldName : DataType`.

3.6 Choosing the right data structure

Different problems call for different data structures. Here's a guide to help you choose:

Scenario	Best structure	Reason
List of test scores	1D Array	Simple list of same-type values
Game board (noughts and crosses)	2D Array	Grid with rows and columns
Seating plan for cinema	2D Array	Rows of seats arranged in grid

Scenario	Best structure	Reason
One student's details	Record	Different data types (name, age, grade)
One product in a shop	Record	Mix of name, price, quantity
All students in a class	Array of Records	Multiple entities, each with multiple fields
Timetable for a week	2D Array	Days (rows) × periods (columns)

Key insight. Use a **1D array** when you have a simple list of similar items. Use a **2D array** when your data naturally forms a grid or table. Use a **record** when you need to group different types of data about a single entity. Use an **array of records** when you have multiple entities, each with the same set of fields.

Worked examples

These examples follow the style of AQA Paper 1 questions. They cover definition questions, code completion, trace tables, and algorithm writing.

Worked Example 1 — Defining a data structure

Question: Which of the following best describes a data structure? [1 mark]

A. A number with a fractional part
B. A value such as a whole number
C. All of the data used and stored within a program
D. An organised collection of values

Answer: D — An organised collection of values.

Exam note. Option A describes a Real/Float. Option B describes an Integer. Option C is too broad. A data structure is specifically an organised way to store and manage a collection of related data values.

Worked Example 2 — Array indexing

Question: An array called `colours` contains the following values:

```
colours = ["Red", "Green", "Blue", "Yellow", "Purple"]
```

(a) State the value of `colours[2]`. [1 mark]
(b) State the index of "Yellow". [1 mark]
(c) Write a statement to change "Purple" to "Orange". [1 mark]

Answers:
(a) **Blue** — Arrays are zero-indexed, so index 2 is the third element.
(b) **3** — Counting from 0: Red(0), Green(1), Blue(2), Yellow(3).
(c) `colours[4] = "Orange"` or `colours[4] ← "Orange"`

Worked Example 3 — Calculating array total

Question: Write an algorithm that calculates and outputs the total of all values stored in an array called `numbers`. [3 marks]

Python answer:

```python
total = 0
for i in range(len(numbers)):
    total = total + numbers[i]
print(total)
```

AQA pseudo-code answer:

```
total ← 0
FOR i ← 0 TO LEN(numbers) - 1
    total ← total + numbers[i]
ENDFOR
OUTPUT total
```

Marking points. (1) Initialise total to 0. (2) Loop through all elements using correct index range. (3) Add each element to total and output result.

Worked Example 4 — Finding maximum value

Question: Write an algorithm that finds and outputs the largest value in an array called `scores`. Do not use any built-in functions. [4 marks]

Python answer:

```
largest = scores[0]
for i in range(1, len(scores)):
    if scores[i] > largest:
        largest = scores[i]
print(largest)
```

Marking points. (1) Initialise largest to first element. (2) Loop through remaining elements. (3) Compare each element to current largest. (4) Update largest if current element is bigger.

Worked Example 5 — 2D array access

Question: A 2D array called `grid` stores the following values:

```
grid = [[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]]
```

(a) State the value of `grid[1][2]`. [1 mark]
(b) State the indices needed to access the value 7. [1 mark]
(c) Write a statement to change the value 5 to 50. [1 mark]

Answers:
(a) **6** — Row 1 is [4, 5, 6], column 2 is the third value (6).
(b) **grid[2][0]** — 7 is in row 2, column 0.
(c) `grid[1][1] = 50` — 5 is in row 1, column 1.

Worked Example 6 — 2D array trace table

Question: Trace through the following algorithm and state the final value of `count`. [3 marks]

```
grid = [[1, 0, 1],
        [0, 1, 0],
        [1, 1, 0]]
count ← 0
FOR row ← 0 TO 2
    FOR col ← 0 TO 2
        IF grid[row][col] = 1 THEN
            count ← count + 1
        ENDIF
    ENDFOR
ENDFOR
OUTPUT count
```

Trace:

Position	Value	count
[0][0]	1	1
[0][1]	0	1
[0][2]	1	2
[1][0]	0	2
[1][1]	1	3
[1][2]	0	3
[2][0]	1	4
[2][1]	1	5
[2][2]	0	5

Answer: The final value of `count` is **5**.
The algorithm counts how many cells contain the value 1.

Worked Example 7 — Game board algorithm

Question: A game uses a row of cells represented as an array. The player starts at position 0 and must reach the end. Cells containing "X" are obstacles. Write an algorithm that checks if position `pos` contains an obstacle. If it does, output "Bad move" and reset `pos` to 0. [3 marks]

Python answer:

```python
if row[pos] == "X":
    print("Bad move")
    pos = 0
```

AQA pseudo-code answer:

```
IF row[pos] = "X" THEN
    OUTPUT "Bad move"
    pos ← 0
ENDIF
```

Worked Example 8 — Sum of 2D array row

Question: Write an algorithm that calculates and outputs the sum of all values in row 0 of a 2D array called `table`. [3 marks]

Python answer:

```python
total = 0
for col in range(len(table[0])):
```

```
    total = total + table[0][col]
print(total)
```

Key point. To access a specific row, fix the row index and loop through the columns. Here, `table[0][col]` accesses each column in row 0.

Worked Example 9 — Completing a record definition

Question: Complete Figure 1 by filling in the gaps to create a record called `Book` with fields `bookName`, `author` and `price`. [3 marks]

```
RECORD _____________
   bookName : String
   _____________ : String
   price : _____________
ENDRECORD
```

Answer:

```
RECORD Book
   bookName : String
   author : String
   price : Real
ENDRECORD
```

Data types. Names and text are String. Prices with decimals are Real. Whole numbers are Integer. True/false values are Boolean.

Worked Example 10 — Creating record variables

Question: Using the Book record definition, write statements to create two book variables: B1 for "The Book Thief" by "M Zusak" costing £9.99, and B2 for "Divergent" by "V Roth" costing £6.55. [2 marks]

AQA pseudo-code answer:

```
B1 ← Book("The Book Thief", "M Zusak", 9.99)
B2 ← Book("Divergent", "V Roth", 6.55)
```

Python answer:

```
B1 = {"bookName": "The Book Thief", "author": "M Zusak", "price
B2 = {"bookName": "Divergent", "author": "V Roth", "price": 6.5
```

Worked Example 11 — Comparing record fields

Question: Write an algorithm that compares the price of B1 and B2, outputs the book name of the most expensive book, or outputs "Neither" if they are the same price. [4 marks]

Python answer:

```
if B1["price"] > B2["price"]:
    print(B1["bookName"])
elif B2["price"] > B1["price"]:
    print(B2["bookName"])
else:
    print("Neither")
```

AQA pseudo-code answer:

```
IF B1.price > B2.price THEN
    OUTPUT B1.bookName
ELSE IF B2.price > B1.price THEN
    OUTPUT B2.bookName
ELSE
    OUTPUT "Neither"
ENDIF
```

Marking points. (1) Compare B1.price and B2.price. (2) Output B1.bookName if B1 is more expensive. (3) Output B2.bookName if B2 is more expensive. (4) Handle equal prices with "Neither".

Worked Example 12 — Student record system

Question: A school stores student data using records. Each student has a name (String), age (Integer), and score (Integer). Write an algorithm that loops through an array of student records and outputs the names of all students who scored more than 80. [4 marks]

Python answer:

```python
for i in range(len(students)):
    if students[i]["score"] > 80:
        print(students[i]["name"])
```

AQA pseudo-code answer:

```
FOR i ← 0 TO LEN(students) - 1
    IF students[i].score > 80 THEN
        OUTPUT students[i].name
```

```
    ENDIF
ENDFOR
```

Marking points. (1) Loop through all students using index. (2) Access the score field correctly. (3) Compare score to 80 using >. (4) Output the name field of matching students.

Worked examples summary

Example	Topic	Marks	Key skill
1	Data structure definition	1	MCQ knowledge
2	Array indexing	3	Zero-based indexing
3	Calculating array total	3	Loop and accumulator
4	Finding maximum value	4	Comparison algorithm
5	2D array access	3	Row/column indexing
6	2D array trace table	3	Nested loop tracing
7	Game board algorithm	3	Condition checking
8	Sum of 2D array row	3	Fixed row iteration
9	Completing record definition	3	Gap-fill syntax
10	Creating record variables	2	Record instantiation
11	Comparing record fields	4	Selection with records
12	Student record system	4	Array of records

Quick reference

Key definitions

Term	Definition
Data structure	An organised collection of related data values.
Array	A data structure that stores multiple values of the same type, accessed by index.

Term	Definition
Index	The position of an element in an array (starting from 0).
Element	A single item stored in an array.
Two-dimensional array	An array of arrays, organised in rows and columns.
Record	A data structure that groups related data of different types under named fields.
Field	A named item of data within a record.

Python syntax reference

Operation	1D Array	2D Array	Record (dict)
Create	`arr = [1, 2, 3]`	`grid = [[1,2], [3,4]]`	`rec = {"a": 1, "b": 2}`
Access	`arr[0]`	`grid[0][1]`	`rec["a"]`
Modify	`arr[0] = 5`	`grid[0][1] = 5`	`rec["a"] = 5`
Length	`len(arr)`	`len(grid)` (rows)	`len(rec)` (fields)

AQA pseudo-code syntax

```
1D ARRAY:
  names ← ["Alice", "Bob", "Charlie"]
  OUTPUT names[0]           → Alice
  names[2] ← "Carol"

2D ARRAY:
  grid ← [[1, 2], [3, 4]]
  OUTPUT grid[0][1]         → 2
  grid[1][0] ← 5

RECORD DEFINITION:
  RECORD Student
      name : String
      age : Integer
      grade : String
  ENDRECORD
```

```
RECORD USAGE:
  s1 ← Student("Alice", 16, "A")
  OUTPUT s1.name            → Alice
  s1.grade ← "A*"

LOOPS WITH ARRAYS:
  FOR i ← 0 TO LEN(names) - 1
      OUTPUT names[i]
  ENDFOR
```

Typical pitfalls

Pitfall 1 — Starting index at 1 instead of 0.
Wrong: `scores[1]` to get the first element.
Right: Array indices start at 0. Use `scores[0]` for the first element.

Pitfall 2 — Accessing beyond array bounds.
Wrong: `scores[5]` when the array has 5 elements.
Right: For an array of length 5, valid indices are 0 to 4. The last element is `scores[len(scores)-1]`.

Pitfall 3 — Wrong order for 2D array indices.
Wrong: `grid[column][row]`
Right: Always row first, then column: `grid[row][column]`.

Pitfall 4 — Confusing record field access.
Wrong: `student[name]` (missing quotes in Python).
Right: Use `student["name"]` in Python or `student.name` in pseudo-code.

Pitfall 5 — Using wrong data types in records.
Wrong: `price : Integer` for a value like 9.99.
Right: Prices with decimals need `price : Real`.

Pitfall 6 — Forgetting to initialise accumulator variables.
Wrong: Using `total` without setting it to 0 first.
Right: Always initialise: `total = 0` before the loop.

Pitfall 7 — Wrong loop range for arrays.
Wrong: `for i in range(len(arr) + 1)` (goes one too far).

Right: Use `range(len(arr))` which gives 0 to length-1.

Pitfall 8 — Confusing arrays and records.
Wrong: Using indices to access record fields.
Right: Arrays use numeric indices; records use field names.

Pitfall 9 — Forgetting nested loops for 2D arrays.
Wrong: Using one loop to process all elements of a 2D array.
Right: Use nested loops — outer for rows, inner for columns.

Pitfall 10 — Missing ENDRECORD in pseudo-code.
Wrong: Leaving record definition unclosed.
Right: Always close with `ENDRECORD` after listing all fields.

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.